

IP + API Secret

AssanPay APIs — Signed Request Manual

Full-security integration with HMAC-SHA256 request signatures. Every request is authenticated with your API key plus a cryptographic signature derived from your API secret.

API Index

POST	/api/payments	H2H Payin API (Async)
POST	/api/payments	H2H Payin Encrypted API
POST	/api/payments/zindigi/confirm	Zindigi OTP Confirm API
POST	/api/sync/payments	H2H Payin API (Sync)
POST	/api/sync/payments	H2H Payin Encrypted API (Sync)
POST	/api/checkout/sessions	Checkout Session API
POST	/api/checkout/sessions	Checkout Session Encrypted API
GET	/api/merchant/status-inquiry?type=payin&orderId={{orderId}}	Payin Status Inquiry
POST	/api/merchant/wallet-payouts	H2H Payout API
POST	/api/merchant/wallet-payouts	H2H Payout Encrypted API
GET	/api/merchant/status-inquiry?type=payout&orderId={{orderId}}	Payout Status Inquiry
GET	/api/merchant/balance/h2h	Balance Check API
WEBHOOK	Callback Signature Verification	Callback Signature Verification
WEBHOOK	Payin Callback	Payin Callback
WEBHOOK	Payout Callback	Payout Callback

Overview

This manual guides you through implementing AssanPay's Host-to-Host (H2H) APIs with IP allowlist + API Secret authentication. Every sensitive endpoint requires HMAC-SHA256 request signatures in addition to the API key and IP check.

- branchCode is required in every request. Create branches from the Branches page, and each branch is approved by admin.
- Your server IPs must be whitelisted before H2H APIs can be used.
- API key and API secret are generated from the Settings page.
- Final response status codes are listed in the Status Codes section at the end of this manual.
- For callback endpoints, return the documented 2xx response to acknowledge receipt and avoid retries.
- Request encryption can use the sub-merchant API secret, but response decryption and callback signature generation must use the main merchant API secret.

Request Signature

Cryptographic verification of every request

A signature is a cryptographic verification value used to confirm that the request was created by an authorized client and that its content has not been altered in transit.

Which API secret to use

Request encryption can be performed using the sub-merchant's API secret. However, response decryption and callback signature generation/verification must always use the main merchant's API secret.

Worked Example

INPUT	SAMPLE VALUE
Method	POST
Path with Query	/v1/payments/create
Timestamp	1712345678
Nonce	550e8400-e29b-41d4-a716-446655440000
Request Body	{"amount":1000,"currency":"PKR"}
API Secret	your_api_secret

Signature Formula

Signature Formula

```
bodyHash = SHA256(requestBody)
canonical = METHOD + "\n" + PATH_WITH_QUERY + "\n" + TIMESTAMP + "\n" + NONCE + "\n"
+ BODY_HASH
signature = Base64(HMAC-SHA256(canonical, API_SECRET))
```

Request Headers

Request Headers

```
X-API-KEY: your_api_key
X-TIMESTAMP: 1712345678
X-NONCE: 550e8400-e29b-41d4-a716-446655440000
X-SIGNATURE: AbCdEf123456ExampleSignature==
Content-Type: application/json
```

Implementation Example

Implementation Example

```
function generateRequestSignature(
  { method, path, timestamp, nonce, rawBody },
  apiSecret
) {
  const bodyString = rawBody || '';

  // Step 1: Body hash (SHA256 hex)
  const bodyHash = crypto.createHash('sha256')
    .update(bodyString)
    .digest('hex');

  // Step 2: Canonical string
  const canonical = `${method}\n${path}\n${timestamp}\n${nonce}\n${bodyHash}`;

  // Step 3: Signature (HMAC SHA256 -> Base64)
  const signature = crypto.createHmac('sha256', apiSecret)
    .update(canonical)
    .digest('base64');

  return { signature, canonical, bodyHash };
}
```

H2H Payin API (Async)

Immediate pending, final status via callback

POST /api/payments

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client
X-TIMESTAMP	Required	Current Unix timestamp in seconds
X-NONCE	Required	Unique random value generated for each request
X-SIGNATURE	Required	Generated request signature

Request Body Parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
amount	number	Required	Transaction amount with up to 2 decimal places (e.g., 100.00)
paymentMethodName	string	Required	Payment Method Name (Easypaisa, JazzCash, Zindigi, QR)
branchCode	string	Required	Merchant branch code
orderId	string	Required	Unique order ID. Max 20 chars. No special characters.
customerContact	string	Required	Valid payer number
customerEmail	string	Required	Valid format email address
callbackUrl	string	Optional	Merchant callback URL for status updates
customerName	string	Optional	Customer name

Zindigi H2H OTP confirmation

If you're using Zindigi H2H, Zindigi sends an OTP to the customer. After initiating the payin, confirm it using POST `/api/payments/zindigi/confirm` with the same headers.

Request Body

Request Body

```
{
  "orderId": "ORD1999",
  "amount": 10,
  "branchCode": "BR-002",
  "paymentMethodName": "JazzCash",
  "customerContact": "03273595453",
  "customerName": "Bilal",
  "customerEmail": "ali@example.com",
  "callbackUrl": ""
}
```

Immediate Response (Pending)

Immediate Response (Pending)

```
{
  "accountNumber": "03273595453",
  "accountTitle": null,
  "amount": 10.00,
  "currency": "PKR",
  "message": "PSP processing",
  "occurredAt": "2026-04-27T18:22:38.593969+05:00",
  "orderId": "Y69ef42cc4839312",
  "paymentId": "T20260427132238593va94qz",
  "qrPayload": null,
  "rail": "MW",
  "reason": "PENDING",
  "redirectUrl": null,
  "reference": null,
  "status": "PENDING",
  "statusCode": "202",
  "success": false,
  "time": "2026-04-27T18:22:38.608523072+05:00",
  "transactionId": null,
  "type": "payin"
}
```

Failure Responses

   AUTH_REQUIRED

```
{
  "success": false,
  "errorCode": "AUTH_REQUIRED",
  "message": "Missing API key"
}
```

   IP_NOT_ALLOWED

```
{
  "success": false,
  "errorCode": "IP_NOT_ALLOWED",
  "message": "IP not allowed"
}
```

   MISSING_AUTH_HEADERS

```
{
  "success": false,
  "errorCode": "MISSING_AUTH_HEADERS",
  "message": "Missing signature headers"
}
```

   INVALID_SIGNATURE

```
{
  "success": false,
  "errorCode": "INVALID_SIGNATURE",
  "message": "Invalid signature"
}
```

   DUPLICATE_ORDER

```
{
  "success": false,
  "errorCode": "DUPLICATE_ORDER",
  "message": "Order ID already exists"
}
```

H2H Payin Encrypted API

AES-256-GCM encrypted payload transmission

POST /api/payments

Signature on encrypted body

The signature must be generated using the final encrypted request body, not the original plaintext payload.

Which API secret to use

Request encryption can be performed using the sub-merchant's API secret. However, response decryption and callback signature generation/verification must always use the main merchant's API secret.

Zindigi H2H OTP confirmation

If you're using Zindigi H2H, Zindigi sends an OTP to the customer. After initiating the payin (encrypted or non-encrypted), confirm it using POST /api/payments/zindigi/confirm. Headers remain the same as the payin you used.

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client
X-TIMESTAMP	Required	Current Unix timestamp in seconds
X-NONCE	Required	Unique random value generated for each request
X-SIGNATURE	Required	Generated request signature

Encrypted Body Parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
encrypted	bool	Required	Must be true to indicate encrypted payload
alg	string	Required	Must be AES/GCM
iv	string	Required	Base64 URL-safe encoded IV (12 bytes)
cipherText	string	Required	Base64 URL-safe encoded encrypted payload + auth tag

How to Prepare an Encrypted Body

1

Create Original Payload

Build the standard JSON payload.

2

Derive Encryption Key

key = HMAC-SHA256('assanpay:req', API_SECRET)

3

Encrypt Payload

AES-256-GCM with random 12-byte IV. Append auth tag to ciphertext.

4

Encode Values

Base64 URL-safe encode the IV and (ciphertext + tag).

5

Build Final Body

Wrap in { encrypted, alg, iv, cipherText }.

Final Encrypted Body

Final Encrypted Body

```
{
  "encrypted": true,
  "alg": "AES/GCM",
  "iv": "vXFGUThmJYdsNrkc",
  "cipherText": "9TdgmZv_Fvj_mecL..."
}
```

Encryption Example

Encryption Example

```
function encryptPayload(payload, apiSecret) {
  const b64url = (buf) =>
    buf.toString('base64')
      .replace(/\+/g, '-')
      .replace(/\//g, '_')
      .replace(/=+$/g, '');

  const key = crypto
    .createHmac('sha256', apiSecret)
    .update('assanpay:req')
    .digest();

  const iv = crypto.randomBytes(12);
  const cipher = crypto.createCipheriv('aes-256-gcm', key, iv);
  const plain = Buffer.from(JSON.stringify(payload), 'utf8');
  const ciphertext = Buffer.concat([cipher.update(plain), cipher.final()]);
  const tag = cipher.getAuthTag();
  const cipherAll = Buffer.concat([ciphertext, tag]);

  return {
    encrypted: true,
    alg: 'AES/GCM',
    iv: b64url(iv),
    cipherText: b64url(cipherAll)
  };
}
```

Decryption Example

Decryption Example

```
function decryptResponse(encryptedBody, apiSecret) {
  const b64urlToBuffer = (str) => {
    str = str.replace(/-/g, '+').replace(/_/g, '/');
    while (str.length % 4) str += '=';
    return Buffer.from(str, 'base64');
  };

  const key = crypto
    .createHmac('sha256', apiSecret)
    .update('assanpay:res')
    .digest();

  const iv = b64urlToBuffer(encryptedBody.iv);
  const cipherData = b64urlToBuffer(encryptedBody.cipherText);
  const tag = cipherData.slice(cipherData.length - 16);
  const ciphertext = cipherData.slice(0, cipherData.length - 16);

  const decipher = crypto.createDecipheriv('aes-256-gcm', key, iv);
  decipher.setAuthTag(tag);
  const decrypted = Buffer.concat([decipher.update(ciphertext), decipher.final()]);

  return JSON.parse(decrypted.toString('utf8'));
}
```

Zindigi OTP Confirm API

Confirm Zindigi H2H payin using OTP

POST /api/payments/zindigi/confirm

Zindigi sends an OTP to the customer. Use this endpoint to confirm the payin. Headers are the same as your payin request (signed mode: signature headers).

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client
X-TIMESTAMP	Required	Current Unix timestamp in seconds
X-NONCE	Required	Unique random value generated for each request
X-SIGNATURE	Required	Generated request signature

Request Body Parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
otp	string	Required	OTP sent to the user by Zindigi (H2H)
orderId	string	Required	Order ID used in the original payin request

Request Body

Request Body

```
{
  "otp": "26676",
  "orderId": "OR1901"
}
```

Success Response

Success Response

```
{
  "accountNumber": "03273595453",
  "accountTitle": "ZINDAGI",
  "amount": 1.00,
  "currency": "PKR",
  "message": "SUCCESS",
  "occurredAt": "2026-04-20T13:20:26.727538+05:00",
  "orderId": "OR1900",
  "paymentId": "T20260420082026727zbtoci",
  "qrPayload": null,
  "rail": "MW",
  "reason": "SUCCESS",
  "redirectUrl": null,
  "reference": "620830256200",
  "status": "SUCCESS",
  "statusCode": "200",
  "success": true,
  "successAt": "2026-04-20T13:20:48.890622377+05:00",
  "transactionId": "620830256200",
  "type": "payin"
}
```

Failure Response & Confirm unsuccessful

● ● ● Failure Response & Confirm unsuccessful

```
{
  "accountNumber": "03273595453",
  "accountTitle": "ZINDAGI",
  "amount": 3.00,
  "currency": "PKR",
  "failedAt": "2026-04-20T13:20:00.375304+05:00",
  "message": "Zindigi confirm unsuccessful",
  "occurredAt": "2026-04-20T13:19:38.823213+05:00",
  "orderId": "OR1899",
  "paymentId": "T202604200819388235zzkpi",
  "qrPayload": null,
  "rail": "MW",
  "reason": "FAILED",
  "redirectUrl": null,
  "reference": null,
  "status": "FAILED",
  "statusCode": "400",
  "success": false,
  "transactionId": null,
  "type": "payin"
}
```

Failure Response & OTP Invalid

Failure Response & OTP Invalid

```
{
  "accountNumber": "03273595453",
  "accountTitle": "ZINDAGI",
  "amount": 1.00,
  "currency": "PKR",
  "failedAt": "2026-04-20T13:22:04.37516+05:00",
  "message": "OTP is Invalid",
  "occurredAt": "2026-04-20T13:21:36.547131+05:00",
  "orderId": "OR1901",
  "paymentId": "T20260420082136547oef4w6",
  "qrPayload": null,
  "rail": "MW",
  "reason": "FAILED",
  "redirectUrl": null,
  "reference": null,
  "status": "FAILED",
  "statusCode": "400",
  "success": false,
  "transactionId": null,
  "type": "payin"
}
```

H2H Payin API (Sync)

Synchronous payin — response returned after user action

POST /api/sync/payments

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client
X-TIMESTAMP	Required	Current Unix timestamp in seconds
X-NONCE	Required	Unique random value generated for each request
X-SIGNATURE	Required	Generated request signature

Request Body Parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
amount	number	Required	Transaction amount with up to 2 decimal places (e.g., 100.00)
paymentMethodName	string	Required	Payment Method Name (Easypaisa, JazzCash, Zindigi, QR)
branchCode	string	Required	Merchant branch code
orderId	string	Required	Unique order ID. Max 20 chars. No special characters.
customerContact	string	Required	Valid payer number
customerEmail	string	Required	Valid format email address
callbackUrl	string	Optional	Merchant callback URL for status updates
customerName	string	Optional	Customer name

Request Body

Request Body

```
{
  "orderId": "ORD1999",
  "amount": 10,
  "branchCode": "BR-002",
  "paymentMethodName": "JazzCash",
  "customerContact": "03273595453",
  "customerName": "Bilal",
  "customerEmail": "ali@example.com",
  "callbackUrl": ""
}
```

Success Response

Success Response

```
{
  "accountNumber": "03273595453",
  "accountTitle": "DEVTECTS PRIVATE LIMITED",
  "amount": 1.00,
  "currency": "PKR",
  "message": "Thank you for Using JazzCash, your transaction was successful.",
  "occurredAt": "2026-04-27T18:25:08.168579+05:00",
  "orderId": "11489245",
  "paymentId": "T20260427132508168ax625l",
  "qrPayload": null,
  "rail": "MW",
  "reason": "SUCCESS",
  "redirectUrl": null,
  "reference": "710348306763",
  "status": "SUCCESS",
  "statusCode": "200",
  "success": true,
  "successAt": "2026-04-27T18:25:14.601495+05:00",
  "transactionId": "710348306763",
  "type": "payin"
}
```

H2H Payin Encrypted API (Sync)

AES-256-GCM encrypted payload — signature on encrypted body

The encrypted version of the H2H Payin Sync API ensures that the request body is securely transmitted using encryption.

Instead of sending the raw JSON payload, the client encrypts the request body using AES-256-GCM. The encrypted payload is then sent to the API along with the standard authentication headers. The signature is generated on the encrypted body, not the raw payload.

Signature on encrypted body

Signature must be generated using the final encrypted request body, not the original payload.

Which API secret to use

Request encryption can be performed using the sub-merchant's API secret. However, response decryption and callback signature generation/verification must always use the main merchant's API secret.

POST /api/sync/payments

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client
X-TIMESTAMP	Required	Current Unix timestamp in seconds
X-NONCE	Required	Unique random value generated for each request
X-SIGNATURE	Required	Generated request signature

Request Body Parameters (Encrypted)

FIELD	TYPE	REQUIRED	DESCRIPTION
encrypted	bool	Required	Must be true to indicate encrypted payload
alg	string	Required	Must be AES/GCM
iv	string	Required	Base64 URL-safe encoded IV (12 bytes)
cipherText	string	Required	Base64 URL-safe encoded encrypted payload + auth tag

How to Prepare an Encrypted Body

1

Create Original Payload

Build the standard JSON payload.

2

Derive Encryption Key

key = HMAC-SHA256('assanpay:req', API_SECRET)

3

Encrypt Payload

AES-256-GCM with random 12-byte IV. Append auth tag to ciphertext.

4

Encode Values

Base64 URL-safe encode the IV and (ciphertext + tag).

5

Build Final Body

Wrap in { encrypted, alg, iv, cipherText }.

Original Payload (before encryption)

Original Payload (before encryption)

```
{
  "orderId": "ORD1999",
  "amount": 10,
  "branchCode": "BR-002",
  "paymentMethodName": "JazzCash",
  "customerContact": "03273595453",
  "customerName": "Bilal",
  "customerEmail": "ali@example.com",
  "callbackUrl": ""
}
```

Final Encrypted Body

Final Encrypted Body

```
{
  "encrypted": true,
  "alg": "AES/GCM",
  "iv": "vXFGUThmJYdsNrkc",
  "cipherText": "9TdgmZv_Fvj_mecL..."
}
```

Encryption Example

Encryption Example

```
function encryptPayload(payload, apiSecret) {
  const b64url = (buf) =>
    buf.toString('base64')
      .replace(/\+/g, '-')
      .replace(/\//g, '_')
      .replace(/=+$/g, '');

  const key = crypto
    .createHmac('sha256', apiSecret)
    .update('assanpay:req')
    .digest();

  const iv = crypto.randomBytes(12);
  const cipher = crypto.createCipheriv('aes-256-gcm', key, iv);
  const plain = Buffer.from(JSON.stringify(payload), 'utf8');
  const ciphertext = Buffer.concat([cipher.update(plain), cipher.final()]);
  const tag = cipher.getAuthTag();
  const cipherAll = Buffer.concat([ciphertext, tag]);

  return {
    encrypted: true,
    alg: 'AES/GCM',
    iv: b64url(iv),
    cipherText: b64url(cipherAll)
  };
}
```

Success Response (Encrypted)

● ● ● Success Response (Encrypted)

```
{
  "encrypted": true,
  "alg": "AES/GCM",
  "iv": "NIIdWfHS6pNe9ow7e",
  "cipherText": "Wquopg9kMpJpkCbGi3Li72EwMBL..."
}
```

Decryption Example

● ● ● Decryption Example

```
function decryptResponse(encryptedBody, apiSecret) {
  const b64urlToBuffer = (str) => {
    str = str.replace(/-/g, '+').replace(/_/g, '/');
    while (str.length % 4) str += '=';
    return Buffer.from(str, 'base64');
  };

  const key = crypto
    .createHmac('sha256', apiSecret)
    .update('assanpay:res')
    .digest();

  const iv = b64urlToBuffer(encryptedBody.iv);
  const cipherData = b64urlToBuffer(encryptedBody.cipherText);
  const tag = cipherData.slice(cipherData.length - 16);
  const ciphertext = cipherData.slice(0, cipherData.length - 16);

  const decipher = crypto.createDecipheriv('aes-256-gcm', key, iv);
  decipher.setAuthTag(tag);
  const decrypted = Buffer.concat([decipher.update(ciphertext), decipher.final()]);

  return JSON.parse(decrypted.toString('utf8'));
}
```

Failure Responses

Encrypted Response (Example)

```
{
  "encrypted": true,
  "alg": "AES/GCM",
  "iv": "NIIdWfHS6pNe9ow7e",
  "cipherText": "Wquopg9kMpJpkCbGi3Li72EwMBL..."
}
```

MISSING_AUTH_HEADERS

```
{
  "success": false,
  "errorCode": "MISSING_AUTH_HEADERS",
  "message": "Missing signature headers",
  "details": null,
  "timestamp": [
    2026,
    4,
    4,
    19,
    5,
    39,
    829246000
  ]
}
```

Checkout Session API

Hosted checkout page for customer-initiated payments

POST /api/checkout/sessions

Generates a unique checkout session URL per transaction. Share this URL with the customer to complete payment on the hosted checkout page.

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client
X-TIMESTAMP	Required	Current Unix timestamp in seconds
X-NONCE	Required	Unique random value generated for each request
X-SIGNATURE	Required	Generated request signature

Request Body Parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
amount	number	Required	Transaction amount up to 2 decimal points
branchCode	string	Required	Merchant branch code
orderId	string	Required	Unique order ID. Max 20 chars. No special characters.
paymentMethodName	string	Optional	Easypaisa, JazzCash, QR, Card
customerContact	string	Required	Valid payer number
customerEmail	string	Required	Valid format email address
callbackUrl	string	Optional	Callback URL for status updates
customerName	string	Optional	Customer name
successUrl	string	Optional	Redirect URL after successful payment
failedUrl	string	Optional	Redirect URL after failed payment
pendingUrl	string	Optional	Redirect URL for pending
expiresInSeconds	number	Optional	Expiry in seconds. Max 600.

Request Body

Request Body

```
{
  "orderId": "ORD1137a9970163",
  "branchCode": "123000",
  "amount": 1.00,
  "paymentMethodName": "jazzcash",
  "customerContact": "03409583505",
  "customerEmail": "buyer@example.com",
  "callbackUrl": "https://merchant.com/webhook",
  "customerName": "Buyer Name",
  "successUrl": "https://merchant.com/success",
  "failedUrl": "https://merchant.com/failed",
  "pendingUrl": "https://merchant.com/pending",
  "expiresInSeconds": 600
}
```

Success Response

Success Response

```
{
  "sessionId": "6a01cafb-8e17-45e3-97ac-37e433fa7bfd",
  "orderId": "ORD1522",
  "paymentId": 111153,
  "checkoutUrl":
    "https://business.assanpay.com/pay?sessionId=6a01cafb-8e17-45e3-97ac-37e433fa7bfd",
  "expiresAt": 1776247695
}
```

Checkout Session Encrypted API

AES-256-GCM encrypted checkout payload — signature required

POST /api/checkout/sessions

Signature required for encrypted checkout

Even in IP whitelist mode, encrypted endpoints require full HMAC signature headers. The signature must be generated using the final encrypted request body, not the plaintext payload.

Which API secret to use

Request encryption can be performed using the sub-merchant's API secret. However, response decryption and callback signature generation/verification must always use the main merchant's API secret.

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client
X-TIMESTAMP	Required	Current Unix timestamp in seconds
X-NONCE	Required	Unique random value generated for each request
X-SIGNATURE	Required	Generated request signature

Encrypted Body Parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
encrypted	bool	Required	Must be true to indicate encrypted payload
alg	string	Required	Must be AES/GCM
iv	string	Required	Base64 URL-safe encoded IV (12 bytes)
cipherText	string	Required	Base64 URL-safe encoded encrypted payload + auth tag

How to Prepare an Encrypted Body

1

Create Original Payload

Build the standard JSON payload.

2

Derive Encryption Key

key = HMAC-SHA256('assanpay:req', API_SECRET)

3

Encrypt Payload

AES-256-GCM with random 12-byte IV. Append auth tag to ciphertext.

4

Encode Values

Base64 URL-safe encode the IV and (ciphertext + tag).

5

Build Final Body

Wrap in { encrypted, alg, iv, cipherText }.

Final Encrypted Body

Final Encrypted Body

```
{
  "encrypted": true,
  "alg": "AES/GCM",
  "iv": "vXFGUThmJYdsNrkc",
  "cipherText": "9TdgmZv_Fvj_mecL..."
}
```

Encryption Example

Encryption Example

```
function encryptPayload(payload, apiSecret) {
  const b64url = (buf) =>
    buf.toString('base64')
      .replace(/\+/g, '-')
      .replace(/\//g, '_')
      .replace(/=+$/g, '');

  const key = crypto
    .createHmac('sha256', apiSecret)
    .update('assanpay:req')
    .digest();

  const iv = crypto.randomBytes(12);
  const cipher = crypto.createCipheriv('aes-256-gcm', key, iv);
  const plain = Buffer.from(JSON.stringify(payload), 'utf8');
  const ciphertext = Buffer.concat([cipher.update(plain), cipher.final()]);
  const tag = cipher.getAuthTag();
  const cipherAll = Buffer.concat([ciphertext, tag]);

  return {
    encrypted: true,
    alg: 'AES/GCM',
    iv: b64url(iv),
    cipherText: b64url(cipherAll)
  };
}
```

Success Response

Success Response

```
{
  "sessionId": "6a01cafb-8e17-45e3-97ac-37e433fa7bfd",
  "orderId": "ORD1522",
  "paymentId": 111153,
  "checkoutUrl":
    "https://business.assanpay.com/pay?sessionId=6a01cafb-8e17-45e3-97ac-37e433fa7bfd",
  "expiresAt": 1776247695
}
```

Payin Status Inquiry

Query the status of a payin transaction

GET /api/merchant/status-inquiry?type=payin&orderId={{orderId}}

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client

No request body required for this endpoint.

Success Response

```
Success Response

{
  "accountNumber": "03273595453",
  "accountTitle": "",
  "amount": 1.00,
  "currency": "PKR",
  "message": "SUCCESS",
  "occurredAt": "2026-04-26T00:25:14.759495+05:00",
  "orderId": "ORD1301",
  "paymentId": "ORD1301",
  "qrPayload": null,
  "rail": "MW",
  "reason": null,
  "redirectUrl": null,
  "reference": "48790687829",
  "status": "SUCCESS",
  "statusCode": "200",
  "success": true,
  "successAt": "2026-04-26T00:25:18.155661+05:00",
  "transactionId": "48271047693",
  "type": "payin"
}
```

Failure Response

Failure Response

```
{
  "accountNumber": "03273595453",
  "accountTitle": "",
  "amount": 1.00,
  "currency": "PKR",
  "failedAt": "2026-04-24T22:02:10.594062+05:00",
  "message": "You did not enter your mobile account pin or approve this transaction from Easypaisa App.",
  "occurredAt": "2026-04-24T22:01:01.153234+05:00",
  "orderId": "T202604241701011530kz12k",
  "paymentId": "T202604241701011530kz12k",
  "qrPayload": null,
  "rail": "MW",
  "reason": "FAILED",
  "redirectUrl": null,
  "reference": "48952091994",
  "status": "FAILED",
  "statusCode": "400",
  "success": false,
  "transactionId": "48952091994",
  "type": "payin"
}
```

H2H Payout API

Initiate wallet or bank payout from merchant system

POST /api/merchant/wallet-payouts

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client
X-TIMESTAMP	Required	Current Unix timestamp in seconds
X-NONCE	Required	Unique random value generated for each request
X-SIGNATURE	Required	Generated request signature

Request Body Parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
amount	number	Required	Transaction amount with up to 2 decimal places
branchCode	string	Required	Merchant branch code
bankCode	string	Required	Bank code representing the destination bank
accountNo	string	Required	Valid receiving account/wallet number
orderId	string	Required	Unique payout order ID. Max 20 chars. No special characters.
callbackUrl	string	Optional	Merchant callback URL for status updates
purpose	string	Optional	Purpose or description of the payout
phone	string	Optional	Beneficiary phone number, if applicable
cnic	string	Optional	Beneficiary CNIC, if applicable

Request Body

Request Body

```
{
  "amount": 1000.00,
  "branchCode": "BR-002",
  "orderId": "ORDPAYOUT1001",
  "bankCode": "59",
  "accountNo": "03273595453",
  "purpose": "Settlement",
  "cnic": "42000000000000",
  "phone": "03330000000",
  "callbackUrl": "https://merchant.com/webhook"
}
```

Success Response — IBFT

Success Response — IBFT

```
{
  "accountNumber": "03273595453",
  "accountTitle": null,
  "amount": 1.00,
  "currency": "PKR",
  "message": "IBFT payout completed successfully",
  "occurredAt": "2026-04-27T18:20:19.333174+05:00",
  "orderId": "WP-3-933829",
  "payoutId": 933829,
  "qrPayload": null,
  "rail": "IBFT",
  "reason": "IBFT payout completed successfully",
  "redirectUrl": null,
  "reference": "710347842452",
  "status": "SUCCESS",
  "statusCode": "200",
  "success": true,
  "successAt": "2026-04-27T18:20:23.020148+05:00",
  "transactionId": "710347842452",
  "type": "payout"
}
```

Success Response — MW (Wallet)

Success Response — MW (Wallet)

```
{
  "accountNumber": "03273595453",
  "accountTitle": null,
  "amount": 1.00,
  "currency": "PKR",
  "message": "MW payout completed successfully",
  "occurredAt": "2026-04-27T18:20:19.333174+05:00",
  "orderId": "WP-3-933830",
  "payoutId": 933830,
  "qrPayload": null,
  "rail": "MW",
  "reason": "MW payout completed successfully",
  "redirectUrl": null,
  "reference": "710347842453",
  "status": "SUCCESS",
  "statusCode": "200",
  "success": true,
  "successAt": "2026-04-27T18:20:23.020148+05:00",
  "transactionId": "710347842453",
  "type": "payout"
}
```

Pending Response

Pending Response

```
{
  "accountNumber": "03273595453",
  "accountTitle": null,
  "amount": 20.00,
  "currency": "PKR",
  "message": "Pending",
  "occurredAt": "2026-04-07T13:08:52.263903+05:00",
  "orderId": "W811",
  "payoutId": 811,
  "qrPayload": null,
  "rail": "IBFT",
  "reason": "Pending",
  "redirectUrl": null,
  "reference": "8101681900",
  "status": "PENDING",
  "statusCode": "202",
  "success": false,
  "time": "2026-04-07T13:08:57.567015072+05:00",
  "transactionId": "8101681900",
  "type": "payout"
}
```

Failed Response — MW (Wallet)

Failed Response — MW (Wallet)

```
{
  "accountNumber": "03273595454",
  "accountTitle": null,
  "amount": 1.00,
  "currency": "PKR",
  "failedAt": "2026-04-27T18:21:14.419835+05:00",
  "message": "Receiver Information is invalid. Kindly recheck User Information via Title Fetch API, before re-executing. Thank you!",
  "occurredAt": "2026-04-27T18:21:10.679714+05:00",
  "orderId": "WP1799",
  "payoutId": 799,
  "qrPayload": null,
  "rail": "MW",
  "reason": "Receiver Information is invalid. Kindly recheck User Information via Title Fetch API, before re-executing. Thank you!",
  "redirectUrl": null,
  "reference": null,
  "status": "FAILED",
  "statusCode": "400",
  "success": false,
  "transactionId": null,
  "type": "payout"
}
```

Failure Responses

DUPLICATE_ORDER

```
{
  "errorCode": "DUPLICATE_ORDER",
  "message": "Order ID already exists",
  "success": false
}
```

VALIDATION_ERROR

```
{
  "errorCode": "VALIDATION_ERROR",
  "message": "orderId: must contain only letters and digits",
  "success": false
}
```

OPERATION_NOT_ALLOWED

```
{
  "errorCode": "OPERATION_NOT_ALLOWED",
  "message": "Insufficient balance for payout",
  "success": false
}
```

INVALID_REQUEST

```
{
  "errorCode": "INVALID_REQUEST",
  "message": "Bank code is required",
  "success": false
}
```

H2H Payout Encrypted API

Encrypted payout with AES-256-GCM

POST /api/merchant/wallet-payouts

Signature on encrypted body

Generate the signature using the final encrypted request body, not the original plaintext payload.

Which API secret to use

Request encryption can be performed using the sub-merchant's API secret. However, response decryption and callback signature generation/verification must always use the main merchant's API secret.

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client
X-TIMESTAMP	Required	Current Unix timestamp in seconds
X-NONCE	Required	Unique random value generated for each request
X-SIGNATURE	Required	Generated request signature

Encrypted Body Parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
encrypted	bool	Required	Must be true to indicate encrypted payload
alg	string	Required	Must be AES/GCM
iv	string	Required	Base64 URL-safe encoded IV (12 bytes)
cipherText	string	Required	Base64 URL-safe encoded encrypted payload + auth tag

Original Payload (before encryption)

```
Original Payload (before encryption)

{
  "amount": 1.00,
  "branchCode": "BR-012",
  "orderId": "ORD-PAYOUT-1919",
  "bankCode": "59",
  "accountNo": "03273595453",
  "purpose": "",
  "cnic": "",
  "phone": "",
  "callbackUrl": "https://merchant.com/webhook"
}
```

Final Encrypted Body

Final Encrypted Body

```
{
  "encrypted": true,
  "alg": "AES/GCM",
  "iv": "vXFGUThmJYdsNrkc",
  "cipherText": "9TdgmZv_Fvj_mecL..."
}
```

Encryption Example

Encryption Example

```
function encryptPayload(payload, apiSecret) {
  const b64url = (buf) =>
    buf.toString('base64')
      .replace(/\+/g, '-')
      .replace(/\//g, '_')
      .replace(/=+$/g, '');

  const key = crypto
    .createHmac('sha256', apiSecret)
    .update('assanpay:req')
    .digest();

  const iv = crypto.randomBytes(12);
  const cipher = crypto.createCipheriv('aes-256-gcm', key, iv);
  const plain = Buffer.from(JSON.stringify(payload), 'utf8');
  const ciphertext = Buffer.concat([cipher.update(plain), cipher.final()]);
  const tag = cipher.getAuthTag();
  const cipherAll = Buffer.concat([ciphertext, tag]);

  return {
    encrypted: true,
    alg: 'AES/GCM',
    iv: b64url(iv),
    cipherText: b64url(cipherAll)
  };
}
```

Decryption Example

Decryption Example

```
function decryptResponse(encryptedBody, apiSecret) {
  const b64urlToBuffer = (str) => {
    str = str.replace(/-/g, '+').replace(/_/g, '/');
    while (str.length % 4) str += '=';
    return Buffer.from(str, 'base64');
  };

  const key = crypto
    .createHmac('sha256', apiSecret)
    .update('assanpay:res')
    .digest();

  const iv = b64urlToBuffer(encryptedBody.iv);
  const cipherData = b64urlToBuffer(encryptedBody.cipherText);
  const tag = cipherData.slice(cipherData.length - 16);
  const ciphertext = cipherData.slice(0, cipherData.length - 16);

  const decipher = crypto.createDecipheriv('aes-256-gcm', key, iv);
  decipher.setAuthTag(tag);
  const decrypted = Buffer.concat([decipher.update(ciphertext), decipher.final()]);

  return JSON.parse(decrypted.toString('utf8'));
}
```

Payout Status Inquiry

Query the status of a payout transaction

GET /api/merchant/status-inquiry?type=payout&orderId={{orderId}}

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client

Success Response

Success Response

```
{
  "accountNumber": "03273595453",
  "accountTitle": null,
  "amount": 1.00,
  "currency": "PKR",
  "message": "IBFT payout completed successfully",
  "occurredAt": "2026-04-26T00:25:14.759495+05:00",
  "orderId": "WP-3-915270",
  "payoutId": 915270,
  "qrPayload": null,
  "rail": "IBFT",
  "reason": null,
  "redirectUrl": null,
  "reference": null,
  "status": "SUCCESS",
  "statusCode": "200",
  "success": true,
  "successAt": "2026-04-26T00:25:18.155661+05:00",
  "transactionId": null,
  "type": "payout"
}
```

Balance Check API

Check merchant balance from your system

GET /api/merchant/balance/h2h

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client
X-TIMESTAMP	Required	Current Unix timestamp in seconds
X-NONCE	Required	Unique random value generated for each request
X-SIGNATURE	Required	Generated request signature

Success Response

Success Response

```
{
  "availableBalance": 8.81,
  "balanceAmount": 8.81,
  "branchesByCode": {
    "122222": {
      "amount": 8.81,
      "branchName": "Test2 submerchnat",
      "ledgerBalance": 8.81,
      "unsettledSuccessfulAmount": 0,
      "unsettledSuccessfulCount": 0
    },
    "112233": {
      "amount": 0.00,
      "branchName": "Test submerchnat",
      "ledgerBalance": 0.00,
      "unsettledSuccessfulAmount": 0,
      "unsettledSuccessfulCount": 0
    }
  },
  "currency": "PKR",
  "payoutFreeze": false,
  "pendingBalance": 0,
  "unsettledSuccessfulAmount": 0,
  "unsettledSuccessfulCount": 0,
  "updatedAt": "2026-05-17T01:00:23.598585+05:00"
}
```

Callback Signature Verification

Verify that webhooks are genuinely from AssanPay

A callback signature is used to verify that the webhook request was sent by an authorized source and that its content has not been tampered with. When a webhook is received, generate a signature from the received values and compare it with the signature sent in the request headers.

- Event ID: sent in header X-Assanpay-Event-Id (unique identifier for the event).
- Timestamp: sent in header X-Assanpay-Timestamp (Unix timestamp in seconds).
- Raw request body: the exact raw body string received in the webhook (do not parse or modify before hashing).
- API Secret: your shared secret key used to verify the signature.

Important: do not modify the body

Use the exact raw request body as received. Do not parse JSON, re-stringify, pretty-print, or change whitespace/newlines before generating the signature.

Which API secret to use

Request encryption can be performed using the sub-merchant's API secret. However, response decryption and callback signature generation/verification must always use the main merchant's API secret.

Verification Flow

1

Extract required headers

Read X-Assanpay-Event-Id, X-Assanpay-Timestamp, and X-Assanpay-Signature from the incoming webhook headers.

2

Build the canonical string

Combine EVENT_ID, TIMESTAMP, and RAW_BODY in this exact order, separated by a newline (\n).

3

Generate HMAC

Compute Base64(HMAC-SHA256(canonical, API_SECRET)).

4

Compare signatures

If generated signature matches the received X-Assanpay-Signature, treat the webhook as valid.

Signature Formula

Signature Formula

```
canonical = EVENT_ID + "\n" + TIMESTAMP + "\n" + RAW_BODY
signature = Base64(HMAC-SHA256(canonical, API_SECRET))
```

Verification Example

Verification Example

```
function generateCallbackSignature(rawBody, headers, apiSecret) {
  if (!apiSecret) throw new Error('Missing API secret');

  const eventId = headers['X-Assanpay-Event-Id'] || '';
  const timestamp = headers['X-Assanpay-Timestamp'] || '';
  const body = rawBody ? String(rawBody) : '';

  const canonical = `${eventId}\n${timestamp}\n${body}`;

  const signature = crypto
    .createHmac('sha256', apiSecret)
    .update(canonical)
    .digest('base64');

  return { canonical, signature };
}

function verifyCallbackSignature(rawBody, headers, apiSecret) {
  const generated = generateCallbackSignature(rawBody, headers, apiSecret);
  const receivedSignature = headers['X-Assanpay-Signature'] || '';
  return generated.signature === receivedSignature;
}
```

Payin Callback

Received webhook for payin status updates. Return any 2xx response to acknowledge receipt.

AssanPay expects any 2xx response against the callback. If a 2xx response is not received, the callback is retried 3 times: first after 15 seconds, then after 30 seconds, then after 60 seconds.

Headers

HEADER	REQUIRED	DESCRIPTION
X-Assanpay-Event-Id	Required	Unique event id
X-Assanpay-Timestamp	Required	Current Unix timestamp in seconds
X-Assanpay-Signature	Required	Generated request signature

Which API secret to use

Request encryption can be performed using the sub-merchant's API secret. However, response decryption and callback signature generation/verification must always use the main merchant's API secret.

Callback Body (Success JSON)

Callback Body (Success JSON)

```
{
  "rail": "MW",
  "type": "payin",
  "amount": 1,
  "reason": "SUCCESS",
  "status": "SUCCESS",
  "message": "SUCCESS",
  "orderId": "ORDq135",
  "success": true,
  "currency": "PKR",
  "successAt": "2026-04-21T20:35:04.628845+05:00",
  "paymentId": "T20260421103504628r4dhubq",
  "qrPayload": null,
  "reference": "48271047693",
  "occurredAt": "2026-04-21T15:35:04.628315+05:00",
  "statusCode": "200",
  "redirectUrl": null,
  "accountTitle": null,
  "accountNumber": "03409583505",
  "transactionId": "48790687829"
}
```

Callback Body (Failed JSON)

Callback Body (Failed JSON)

```
{
  "rail": "MW",
  "type": "payin",
  "amount": 1,
  "reason": "SYSTEM ERROR",
  "status": "FAILED",
  "message": "SYSTEM ERROR",
  "orderId": "ORDq135",
  "success": false,
  "currency": "PKR",
  "failedAt": "2026-04-21T20:35:04.628845+05:00",
  "paymentId": "T20260421103504628r4dhubq",
  "qrPayload": null,
  "reference": "48790687829",
  "occurredAt": "2026-04-21T15:35:04.628315+05:00",
  "statusCode": "400",
  "redirectUrl": null,
  "accountTitle": null,
  "accountNumber": "03409583505",
  "transactionId": "48790687829"
}
```

Callback Body (Encrypted)

Callback Body (Encrypted)

```
{
  "encrypted": true,
  "alg": "AES/GCM",
  "iv": "j6mKSr9XrvmoLKD5",
  "cipherText": "BCRgnjSPoPuHF1w4tT9DJ58....."
}
```

Payout Callback

Received webhook for payout status updates. Return any 2xx response to acknowledge receipt.

AssanPay expects any 2xx response against the callback. If a 2xx response is not received, the callback is retried 3 times: first after 15 seconds, then after 30 seconds, then after 60 seconds.

Headers

HEADER	REQUIRED	DESCRIPTION
X-Assanpay-Event-Id	Required	Unique event id
X-Assanpay-Timestamp	Required	Current Unix timestamp in seconds
X-Assanpay-Signature	Required	Generated request signature

Which API secret to use

Request encryption can be performed using the sub-merchant's API secret. However, response decryption and callback signature generation/verification must always use the main merchant's API secret.

Callback Body (Success JSON)

Callback Body (Success JSON)

```
{
  "rail": "IBFT",
  "type": "payout",
  "amount": 1,
  "reason": null,
  "status": "SUCCESS",
  "message": "IBFT payout completed successfully",
  "orderId": "WP-3-933829",
  "success": true,
  "currency": "PKR",
  "payoutId": 933829,
  "qrPayload": null,
  "reference": "710347842452",
  "successAt": "2026-04-27T18:20:19.333175+05:00",
  "occurredAt": "2026-04-27T18:20:19.333174+05:00",
  "statusCode": "200",
  "redirectUrl": null,
  "accountTitle": null,
  "accountNumber": "03273595453",
  "transactionId": "710347842452"
}
```

Callback Body (Failed JSON)

Callback Body (Failed JSON)

```
{
  "rail": "IBFT",
  "type": "payout",
  "amount": 1,
  "reason": "No response from JazzCash IBFT payment",
  "status": "FAILED",
  "message": "No response from JazzCash IBFT payment",
  "orderId": "WP-1-911658",
  "success": false,
  "currency": "PKR",
  "failedAt": "2026-04-21T13:27:56.701022+05:00",
  "payoutId": 911658,
  "qrPayload": null,
  "reference": null,
  "occurredAt": "2026-04-21T13:27:56.701021+05:00",
  "statusCode": "400",
  "redirectUrl": null,
  "accountTitle": null,
  "accountNumber": "01720104776754",
  "transactionId": "WP-1-911658"
}
```

Callback Body (Encrypted)

Callback Body (Encrypted)

```
{
  "encrypted": true,
  "alg": "AES/GCM",
  "iv": "j6mKSr9XrvmoLKD5",
  "cipherText": "BCRgnjSPoPuHF1w4tT9DJ58....."
}
```

Status Codes

Reference for response status codes



200
Status Code

Success



202
Status Code

Pending



400
Status Code

Failed



404
Status Code

Unknown

Payout Bank Code File

Reference PDF and payout bank code mapping

Use this reference file when selecting or validating bankCode values for payout requests.

- Open the Payout Bank Code File (PDF).

Bank Codes

BANK CODE	BANK NAME
96	Advans Microfinance Bank
72	Al Baraka Islamic Bank Limited
43	Bank AlFalah Limited
48	Allied Bank Limited
76	Apna Microfinance Bank
47	Askari Commercial Bank Limited
87	Bank of Khyber
44	Bank Islami Pakistan Limited
67	Burj Bank Limited
57	Citi Bank

56	Dubai Islamic Bank Pakistan Limited
86	FINCA
97	Finja
90	First Women Bank
52	Faysal Bank Limited
49	Habib Bank Limited
77	Habib Metropolitan Bank
95	ICBC
75	JS Bank
51	KASB Bank
79	MCB Bank Limited
100	MCB Arif Habib
91	MCB Islamic Bank
66	Meezan Bank
10	Mobilink Microfinance Bank
99	NayaPay
89	National Bank of Pakistan
65	NIB Bank
88	NRSP Bank Fori Cash
55	Paymax
11	Sadapay
64	Standard Chartered Bank
71	Samba Bank
68	Silkbank
73	Sindh Bank
61	Soneri Bank Limited

58	Summit Bank
12	TAG
60	United Bank Limited
85	Upaisa
101	ZTBL
59	EasyPaisha
000	JazzCash
340	AIK